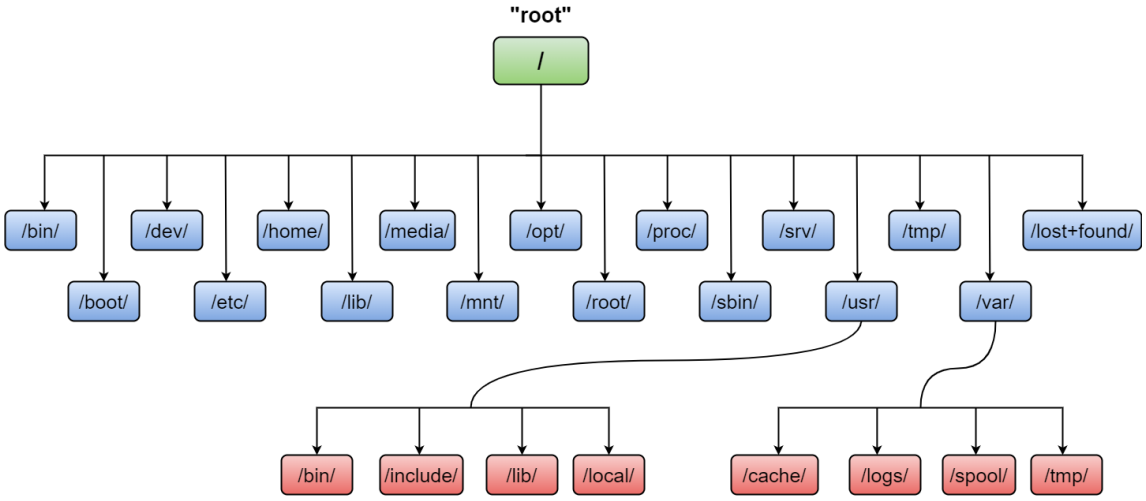
Introduction to Linux OS

By Mohamed Gamal

Exploring the Filesystem

The file structure is nothing but how an individual computer organizes its data and stores it. Linux file structure is somehow different than the Windows and Macintosh operating systems.

In the Linux operating system Filesystem Hierarchy Standard (FHS), all files and directories appear under the “root” (/) directory, even if they are stored on different physical or virtual devices.



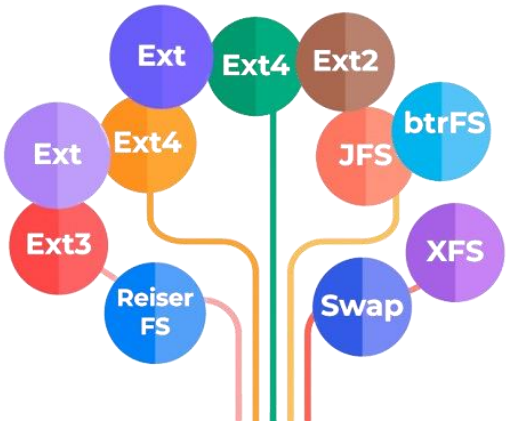
root (UID 0)

Dir	Description
/	The directory called "root." It is the starting point for the file system hierarchy. Note that this is not related to the root, or superuser, account.
/bin	Binaries and other executable programs.
/etc	System configuration files.
/home	Home directories.
/opt	Optional or third party software.
/tmp	Temporary space, typically cleared on reboot.
/usr	User related programs.
/var	Variable data, most notably log files.

Comprehensive Directory Listing

Dir	Description
/	The directory called "root." It is the starting point for the file system hierarchy. Note that this is not related to the root, or superuser, account.
/bin	Binaries and other executable programs.
/boot	Files needed to boot the operating system.
/cdrom	Mount point for CD-ROMs.
/cgroup	Control Groups hierarchy.
/dev	Device files, typically controlled by the operating system and the system administrators.
/etc	System configuration files.
/export	Shared file systems. Most commonly found on Solaris systems.
/home	Home directories.
/lib	System Libraries.
/lib64	System Libraries, 64 bit.
/lost+found	Used by the file system to store recovered files after a file system check has been performed.
/media	Used to mount removable media like CD-ROMs.
/mnt	Used to mount external file systems.
/opt	Optional or third party software.
/proc	Provides information about running processes.
/root	The home directory for the root account.
/sbin	System administration binaries.
/selinux	Used to display information about SELinux.
/srv	Contains data which is served by the system.
/srv/www	Web server files.
/srv/ftp	
/sys	Used to display and sometimes configure the devices and busses known to the Linux kernel.
/tmp	Temporary space, typically cleared on reboot. This directory can be used by the OS and users alike.
/usr	User related programs, libraries, and documentation. The sub-directories in /usr relate to those described above and below.
/usr/bin	Binaries and other executable programs.
/usr/lib	Libraries.
/usr/local	Locally installed software that is not part of the base operating system.
/usr/sbin	System administration binaries.
/var	Variable data, most notably log files.
/var/log	Log files.

Types of Linux File Systems:



File Naming in Linux:

- In any Linux-based system, you do not have to specify the file extension when creating a file.
- However, file extensions are useful for identifying the type of content or for associating the file with the appropriate application.
- File names may be up to 255 characters.
- Avoid special characters as > < ? * # '
- File names are case sensitive (e.g., file1 ≠ FILE1)

Listing All User Commands

To list all the commands available to the user in the current shell session you can use

```
$ compgen -c    or    $ man -k .
```

Locating Applications & Commands (WH Commands)

Depending on the specifics of your particular distribution's policy, programs and software packages can be installed in various directories. In general, executable programs and scripts should live in the **/bin**, **/usr/bin**, **/sbin**, **/usr/sbin** directories, or somewhere under **/opt**. They can also appear in **/usr/local/bin** and **/usr/local/sbin**, or in a directory in a user's account space, such as **/home/mg/bin**.

One way to locate programs/commands is to employ the **which** utility.

Example Usage	
`$ which firefox` `/usr/bin/firefox`	`$ which cat` `/usr/bin/cat`

If **which** does not find the program, **whereis** is a good alternative because it looks for packages in a broader range of system directories:

Example Usage
`$ whereis firefox` `firefox: /usr/bin/firefox  /usr/lib64/firefox  /usr/share/man/man1p/firefox.1p.gz`
`$ where is cat` `cat: /usr/bin/cat  /usr/share/man/man1/cat.1.gz  /usr/share/man/man1p/cat.1p.gz`

Environment Variables

The **PATH** environment variable contains a list of directories that contain executable commands.

```
$ echo $PATH
```

When you type in a command at the prompt and press Enter, that command will be searched for in the directories in your `$PATH`.

For example, `/bin` will be searched first, if the command is found it will be executed. If it is not found, then `/usr/bin` will be searched and so on.

If no executable command is found that matches your request, you will be politely told that it cannot be found.

Common Environment Variables

`$ env` or `$ printenv`

Variable	Description
EDITOR	The program to run to perform edits
HOME	The Home directory of the user
LOGNAME	The login name of the user.
MAIL	The location of the user's local inbox
OLDPWD	The previous working directory.
PATH	A colon separated list of directories to search for commands.
PAGER	This program may be called to view a file.
PS1	The primary prompt string.
PS2	The secondary prompt string.
PWD	The current working directory
USER	The username of the user.

Shell Customization:

```
PS1="<\t \u@\h \w>\$ "
PS2="line continued: "
```

Link 1 Link 2

Symbol	Description
\t	The current time in 24-hour HH:MM:SS format
\u	The username of the current user
\h	The hostname up to the first '.'
\w	The current working directory, with $HOME abbreviated with a tilde
\$	If the effective UID is 0, a #, otherwise a $
...	...

Extra: Oh My Posh

Accessing Directories (Folders)

When you first log into a system or open a terminal, the default directory should be your home directory. You can see the exact location by typing `echo $HOME`.

Directories are simply containers for files and other directories. They provide a tree like structure for organizing the system. Directories can be accessed by their name and they can also be accessed using a couple of shortcuts.

Linux uses the symbols `.` and `..` to represent directories.

- Think of `.` as "this directory"
- and `..` and "the parent directory"

Symbol	Description	Examples

`.`	This (or current) directory.	`$ pwd`
		`$ cd /var/tmp`
		`$ cd .`
		`$ cd ..`

`..`	The parent directory.	
`/`	Directory separator.	

--- Navigating and Managing Directories ---

Command	Usage
`pwd`	Displays the current working directory.
`cd`	Change to another directory. **e.g.**, pushd /tmp popd
`pushd`	
`popd`	
`xdg-open .`	Open the current working directory in a graphical user interface (GUI) file manager.
`ls  or ll`	List the contents of the current working directory.
`tree`	Displays a tree view of the filesystem.

--- Viewing and Managing Files ---

Command	Usage
`touch`	Creates an empty file or updates the timestamp of an existing file.
`cat`	Used for viewing files that are not very long; it does not provide any scroll-back.
`tac`	Used to look at a file backwards, starting with the last line.
`less`	Used to view larger files because it is a paging program. It pauses at each screen full of text, provides scroll-back capabilities, and lets you search and navigate within the file.
`more`	Used to view the contents of a file one screen at a time. It pauses after displaying each page of text but has limited navigation capabilities compared to `less`. You can scroll forward but not backward, making it useful for quick file viewing.
`head`	Used to print the first 10 lines of a file by default. You can change the number of lines by doing -n 15 or just -15 if you wanted to look at the last 15 lines instead of the default.
`tail`	The opposite of head; by default, it prints the first 10 lines of a file. Use **`$ tail -f <file>`** to view changes that are being made in real time to a file.
`wc`	Displays the number of lines, words, and characters in a file.
`stat`	Used to display detailed information about files and directories.
`file`	Used for determining the type of a file by examining its content rather than the file extension. **`$ file *`** **`$ file script.sh`**

--- Creating, Managing, and Removing Files and Directories ---

Command	Usage
`mkdir`	Creates a directory under the current directory.
`rmdir`	Removes an empty directory.
`rm`	Removes files or directories. Use the -r option to remove a directory and its contents recursively.
`cp`	Copies files or directories.

`mv`	Moves or renames files or directories.
	`$ mv file1.txt /home`
	`$ mv file1.txt file2.txt file3.txt /home`
	`$ mv file1.txt file1_renamed.txt`

`du -ah`	Estimates and displays the disk space usage of files and directories.
`!!`	Run the last recent command.

File Globbing (Meta Character)

- **Asterisk (*):** represents 0 or more characters, except leading (.)
- **Question mark (?):** represents any single character except the leading (.)

```
$ touch file{1..3}.txt

$ ls file?.txt

$ ls fileB?.txt

$ ls file.t?t

$ ls -d dir?

$ cp file?.txt /backup
```